IBM Innovation Centre for Education

NCS Nihon Communication Solutions

Yenepoya Institute of Arts Science Commerce and Management

Low Level Design

on

Develop a virtual museum to explore artifacts in 3D

Student Name: Arjun V Sugesh

Industry Mentor: Abhimitra

Guided by: Thejaswini Maam

Table ofContents

1. Introduction.----------3
   1.1. Scope of thedocument.----------3 1.2. Intended Audience----------3 1.3. System overview.----------3
2. System Design.----------4
   2.1. Application Design----------4
   2.2. Process Flow.----------4
   2.3. Information Flow.----------5
   2.4. Components Design----------5
   2.5. Key Design Considerations----------5
   2.6. API Catalogue.----------5
3. Data Design.----------6
   3.1. Data Model----------6
   3.2. Data Access Mechanism----------6
   3.3. Data Retention Policies----------7
   3.4. Data Migration----------7
4. Interfaces----------7
5. State and Session Management----------7
6. Caching----------7
7. Non-Functional Requirements----------8
   7.1. SecurityAspects----------8
   7.2. PerformanceAspects----------8
8. References----------8

1. Introduction

The Low Level Design describes the internal working of the Virtual Museum system. It focuses on implementation details such as modules, classes, methods, and data handling. The system is developed using **Unity** and this document helps developers understand how each component works at a detailed level.

1.1 Scope of the Document

This document provides detailed design information required for implementing the system.
Includes:

- Module-level design
- Class structures
- Data handling logic

1.2 Intended Audience

This document is intended for technical team members involved in development.
Audience:

- Developers
- Testers
- System designers

1.3. System Overview

The system is a 3D application where users can navigate and interact with artifacts. The LLD explains how internal components work together to achieve this functionality.

2. System Design

The system is divided into modules, each responsible for a specific task. These modules communicate with each other to provide complete functionality.

Modules:

- Navigation
- Interaction
- UI
- Data management

2.1. Application Design

The application follows a modular and object-oriented design approach. Each feature is implemented using separate scripts and components.

Design approach:

- Script-based components
- Reusable classes
- Event-driven interaction

2.2. Process Flow

The process defines how the system executes user actions step by step.

Steps:

1. User moves in the environment
2. User selects an artifact
3. System detects object
4. Data is retrieved
5. UI displays information

2.3. Information Flow

This section describes how data moves within the system.

Flow:

- Input → Navigation module
- Interaction → Object detection
- Data → UI display

2.4. Components Design

The system consists of multiple components working together.
Components:

- Player controller
- Interaction controller
- UI manager
- Data manager
- Artifact objects

2.5. Key Design Considerations

The system must ensure efficiency and usability.
Considerations:

- Smooth performance
- Optimized scripts
- Easy interaction

2.6. API Catalogue (Optional)

APIs are used if backend integration is present.
APIs:

- GET /artifacts
- GET /artifact/{id}

3. Data Design

The system manages artifact data in a structured format for efficient access.
Data includes:

- Artifact ID
- Name
- Description
- Model path

3.1 Data Model

Defines structure of stored data.
Includes:

- Artifact entity
- Attributes and relationships

3.2 Data Access Mechanism

Defines how data is retrieved.
Methods:

- JSON files
- API calls

3.3. Data Retention Policies

Defines how long data is stored.
Policies:

- Artifact data → permanent
- Session data → temporary

3.4 Data Migration

Handles updates and additions.
Supports:

- Adding new artifacts
- Updating data

4. Interfaces

Interfaces define communication between components and users.

5. State and Session Management

The system tracks user activity during runtime.
Tracks:

- User position
- Current state
- Active session

6. Caching

Caching improves system performance by storing frequently used data.
Used for:

- Preloading assets
- Reducing load time

7. Non-Functional Requirements

The system must meet quality standards.
Requirements:

- Performance
- Security
- Scalability
- Usability

8. References

This design is based on technical and design standards.
Sources:

- Unity documentation
- Software design principles